The Journal of Systems and Software 226 (2025) 112384

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

Poisoned source code detection in code models☆

Ehab Ghannoum *, Mohammad Ghafari

Technische Universität Clausthal, Germany

ARTICLE INFO

Dataset link: https://doi.org/10.5281/zenodo.14871552

Keywords:
Poisoned code detection
Source code poisoning
Adversarial machine learning

ABSTRACT

Deep learning models have gained popularity for conducting various tasks involving source code. However, their black-box nature raises concerns about potential risks. One such risk is a poisoning attack, where an attacker intentionally contaminates the training set with malicious samples to mislead the model's predictions in specific scenarios. To protect source code models from poisoning attacks, we introduce CodeGarrison (CG), a hybrid deep-learning model that relies on code embeddings to identify poisoned code samples. We evaluated CG against the state-of-the-art technique ONION for detecting poisoned samples generated by DAMP, MHM, ALERT, as well as a novel poisoning technique named CodeFooler. Results showed that CG significantly outperformed ONION with an accuracy of 93.5%. We also tested CG's robustness against unknown attacks, achieving an average accuracy of 85.6% in identifying poisoned samples across the four attacks mentioned above.

1. Introduction

In recent years, with the growing availability of open-source software and its data, deep learning (DL) models have proven highly effective for analyzing software systems (Wang et al., 2020; Liu et al., 2021; Firouzi et al., 2024; Jiang et al., 2021; Bruni et al., 2025). Nevertheless, these models are subject to poison attacks, which compromise the integrity of DL models, i.e., causing them to behave incorrectly or unpredictably under certain conditions (Gu et al., 2019; Jin et al., 2022). For example, a poisoned model for code completion might suggest insecure or incorrect code patterns when specific sequences are provided as input. In this paper, we focus on a specific type of poison attack known as *data poisoning*. In this attack, an attacker includes poisoned samples into a model's training data and poisons the model so that the model functions normally with clean inputs but produces targeted erroneous results with inputs embedded with specific triggers. From here forth, when we refer to poison attacks, we specifically mean data poisoning.

Poisoning attacks in DL models for source code processing are a significant threat. These models are often trained on datasets sourced from public development platforms (e.g., GitHub and StackOverflow) or derived from publicly available benchmarks like CodeXGLUE (Lu et al., 2021). This opens the door for malicious actors to introduce harmful threats disguised as legitimate repositories or datasets, thereby corrupting the training data. Models trained on such compromised datasets may function correctly under normal conditions, but once attackers activate hidden backdoors, they can behave differently, such as evading defect detection or degrading system performance.

There are several poison attack strategies for source code, including DAMP (Yefet et al., 2020), MHM (Zhang et al., 2020), ALERT (Yang et al., 2022), and more general methods like the CodeFooler algorithm (Jin et al., 2019). These techniques generate poisoned samples that are operationally and naturally similar to benign samples; however, they manipulate the code model predictions. These techniques were developed to target a broader range of code models, encompassing pre-trained models trained on vast amounts of code data beforehand, allowing them to learn powerful representations of code structure and functionality. This pre-training provides a strong foundation for various tasks, including code completion, bug detection, and code analysis tools such as CodeBERT and GraphCodeBERT for MHM and ALERT and models such as Code2Vec, GGNN, and GNN-FiLM for DAMP.

Traditional approaches for detecting poisoned samples are often ineffective (Shafahi et al., 2018; Carlini and Wagner, 2017). These methods rely on the source code itself and fail when poisoned samples appear normal while their underlying embeddings have been altered. Consequently, deep source code processing models remain highly vulnerable to poison attacks (Li et al., 2024). Detecting poisoned code is therefore crucial for maintaining the integrity of code models and fostering trust in the system's outputs.

We present CodeGarrison (CG), a hybrid model leveraging code embeddings to detect poisoned code samples within training datasets.

☆ Editor: Prof W. Eric Wong.

* Corresponding author.
E-mail addresses: ehab.ghannoum@tu-clausthal.de (E. Ghannoum), mohammad.ghafari@tu-clausthal.de (M. Ghafari).

https://doi.org/10.1016/j.jss.2025.112384
Received 30 September 2024; Received in revised form 20 January 2025; Accepted 11 February 2025
Available online 12 March 2025
0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

CG is not restricted to a specific task; instead, it identifies poisoned code samples that manipulate a code model's predictions, regardless of whether the model is applied to downstream tasks such as code recommendation, code summarization, or other applications. Particularly, we address the following research questions (RQs):

- **RQ**$_1$: Is CG effective in detecting poisoned code samples? We showed that CG successfully identifies poisoned code samples generated by DAMP, MHM, ALERT, and a new poisoning technique called CodeFooler with an accuracy of 93.1%, 95.5%, 94.9%, and 90.3%, respectively. By contrast, the state-of-the-art ONION model (Qi et al., 2021) achieved a lower accuracy of 91.1%, 90.8%, 91.1%, and 89.3% for the same attacks.
- **RQ**$_2$: How does CG perform against unseen poison attacks? We iteratively removed all poisoned samples generated by one technique from the training data and evaluated the performance of CG against the removed samples. The experimental results demonstrate that CG accurately detects the unseen poisoned code samples generated by DAMP, MHM, ALERT, and CodeFooler with an accuracy of 92.5%, 95.1%, 93.8%, and 60.9%, respectively.
- **RQ**$_3$: How does each feature impact CG's ability to differentiate between poisoned and clean samples? We trained CG on each feature set separately and assessed their impact on the model's performance. We found that combining embeddings from Code2Vec, CodeBERT, and FastText (i.e., using only embeddings) produces the highest accuracy at 95.0%, surpassing the performance achieved when all features, including Code2Vec name predictions and scoring, were utilized (94.0%). In contrast, when each embedding was used individually, the accuracy dropped to 92.5% for CodeBERT, 81.6% for Code2Vec, and 79.2% for FastText.

In summary, we developed CG, a hybrid deep-learning model that relies on code embeddings to identify poisoned source code samples. We showed that CG outperformed ONION, the only publicly available state-of-the-art model for poison detection. Additionally, we found that CG is effective against poisoned samples generated by CodeFooler, a new code poisoning technique that we developed based on CodeFooler. We also investigated CG's capability to detect unseen poison attacks, which is very important for the adoption of poison detection models in the real world. CG can be integrated into development pipelines in two key ways: as a preprocessing module to clean training datasets by removing poisoned code before model fine-tuning, and as a real-time module to analyze code snippets within an IDE, instantly flagging potential poisoning triggers.

The CG model, CodeFooler, our datasets, and experimentation results are publicly available to support further research on this topic.[1]

The rest of this paper is organized as follows. We make an overview of related work in Section 2. We introduce the existing code poisoning attacks in Section 3. The data preparation will be addressed in Section 4. We introduce our poisoned code detection approach in Section 5 and evaluate it in Section 6. We explain the threats to the validity of this study in Section 7. We conclude this paper in Section 8.

2. Related work

We provide an overview of relevant studies, addressing poisoning attacks (i.e., model corruption) and their detection methods (i.e., model protection).

2.1. Model corruption

Cotroneo et al. (2024) explores the security vulnerabilities of AI-based code generators through a targeted data poisoning attack, demonstrating that neural machine translation (NMT) models can generate unsafe code when even a small portion of the training data is maliciously altered. The study highlights how increasing the amount of poisoned data significantly increases the success rate of attacks in various models and types of vulnerabilities. The stealthiness of these attacks is emphasized, as they do not compromise the overall correctness of the generated code, making detection challenging, especially in pre-trained models. Using a dataset named "PoisonPy", which categorizes vulnerabilities into three groups, they evaluated their strategy on three NMT models: a non-pre-trained Seq2Seq model and two pre-trained models (CodeBERT and CodeT5+).

Yang et al. (2024) introduce Adversarial Feature as Adaptive Backdoor (AFRAIDOOR), a stealthy backdoor attack framework targeting code models. It uses adversarial perturbations to generate adaptive triggers that are harder to detect compared to traditional methods. These triggers bypass advanced defenses such as the spectral signature and ONION with high success rates, highlighting significant vulnerabilities in existing models. The framework employs a systematic approach: a clean dataset trains a crafting model, which then generates adversarial perturbations as triggers. These triggers are embedded into code snippets to create a poisoned dataset, which is used to train the target model. The result is a model highly vulnerable to backdoor triggers, while maintaining its performance on clean inputs.

Chen et al. (2020) propose a general NLP backdoor attack framework called BadNL, which introduces innovative attack methods like BadChar, BadWord, and BadSentence, each with both basic and semantic-preserving variants. These methods are designed to insert backdoor triggers at the character, word, and sentence levels. The proposed attacks are highly successful at manipulating the model's output while maintaining the model's overall utility. Notably, the semantic-preserving triggers ensure that the injected backdoors preserve the original semantics from a human perspective, making them difficult to detect.

Ji et al. (2018) present a broad class of "model-reuse attacks", demonstrating how maliciously crafted primitive models can manipulate host machine learning systems to misbehave on targeted inputs in a highly predictable manner. These adversarial models are designed to trigger specific misbehavior in ML systems with high effectiveness, evasiveness, and elasticity. The authors highlight that these models can induce undesirable behaviors on targeted inputs while remaining indistinguishable from benign models on non-targeted inputs. Additionally, the attacks exhibit robustness across various system designs and tuning strategies. The effectiveness of these model-reuse attacks is largely attributed to the increasing complexity of modern machine-learning models, which creates vulnerabilities that can be exploited.

Wan et al. (2022) show that deep-learning-based code search models are vulnerable to data poisoning attacks, specifically backdoor attacks that can manipulate the ranking of search results. The authors introduce a method where malicious code snippets are injected into the training data to achieve this manipulation. Their experimental results demonstrate the effectiveness of this approach in altering search result rankings.

Schuster et al. (2020) show that neural code auto-completers are vulnerable to poisoning attacks, allowing attackers to influence the auto-completer suggestions in specific, attacker-chosen contexts without significantly altering its behavior in other contexts. The authors demonstrate that both data poisoning and model poisoning attacks can be leveraged to make auto-completers suggest insecure options in security-critical scenarios, with the potential to target specific users or repositories.

Aghakhani et al. (2023) find that the COVERT and TROJANPUZZLE attacks pose serious threats to code-suggestion models by evading

[1] https://doi.org/10.5281/zenodo.14871552

conventional defenses like static analysis. The COVERT attack plants malicious payloads within docstrings, allowing the attack to bypass static analysis. Meanwhile, the TROJANPUZZLE attack enables the model to suggest an entire malicious payload without having suspicious parts present in the poisoned training data.

Li et al. (2024) present CodePoisoner, a sophisticated poisoning attack framework targeting deep learning models for source code processing tasks such as defect detection, clone detection, and code repair. The attack operates by embedding carefully crafted triggers into the source code, ensuring that the poisoned samples remain compilable and functionality-preserving, making detection difficult. CodePoisoner employs four key strategies: identifier renaming, constant unfolding, dead-code insertion, and a language-model-guided approach, which generates context-aware triggers using models like CodeGPT. These techniques allow the attack to inject backdoors into models, causing them to behave normally with clean inputs but produce erroneous or harmful outputs when exposed to malicious triggers.

2.2. Model protection

Steinhardt et al. (2017) propose a framework for analyzing data poisoning attacks against machine learning systems, focusing on defenders that first perform outlier removal followed by empirical risk minimization. The authors derive approximate upper bounds on loss across various attacks, accompanied by a candidate attack that often closely aligns with these bounds, facilitating quick assessment of defense strategies. Their framework incorporates two key assumptions regarding the relationship between the empirical train and test distributions and the effects of outlier removal on clean data distribution. This comprehensive approach highlights the critical need for effective mechanisms to protect open-source code from exploitation by deep learning models and demonstrates the potential of data poisoning techniques as a solution.

Qi et al. (2021) propose ONION, a pre-trained language model that detects poisoned samples in deep learning models through uncovering words in test samples that serve as backdoor triggers. It calculates the perplexity of a sentence with and without each word. Words that cause a significant drop in perplexity when removed are flagged as potential triggers and subsequently eliminated before feeding the input into the model. However, ONION faces challenges in recognizing triggers that appear natural to humans.

Sun et al. (2022) introduce CoProtector, a framework designed to safeguard open-source code from unauthorized use by deep learning models through data poisoning techniques. Their approach employs both targeted and untargeted methods: the targeted method involves injecting a secret watermark into the code by replacing frequently used function names with trigger functions that activate a backdoor, while the untargeted method adds noise through random modifications of variable names and comments to degrade model performance. Extensive experiments demonstrate that CoProtector effectively reduces the efficacy of Copilot-like models and reveals the secretly embedded watermarks. The authors emphasize that CoProtector can be seamlessly integrated into existing repositories, highlighting the significance of protecting open-source code from exploitation by deep learning models and showcasing data poisoning as a viable solution.

Razmi and Xiong (2023) shows that recent protections against data poisoning attacks, such as k-nearest neighbors or centroid-based outlier detection, rely heavily on the availability of clean data to train the defense models effectively. Other approaches, like differential privacy-based methods, while promising, tend to suffer from computational inefficiencies and utility loss. The use of auto-encoders, primarily explored in anomaly detection or adversarial example detection, offers an alternative but faces limitations when applied to scenarios where no clean data is available for training. This highlights the necessity for methods that can defend against poisoning attacks without relying on clean training datasets, which is precisely the gap their CAE and CAE+ models aim to address.

Li et al. (2024) introduce CodeDetector, a defense mechanism against poisoning attacks targeting deep learning models in source code processing tasks. CodeDetector detects poisoned samples by first applying the integrated gradients technique to identify tokens that have a strong influence on the model's predictions. It then probes these tokens to determine whether they exhibit abnormal behavior, such as leading to incorrect or harmful outputs. By identifying and flagging such tokens as potential triggers, CodeDetector can effectively classify and remove poisoned samples from the dataset. This approach allows CodeDetector to defend against a range of poisoning strategies, including those that use static or context-aware triggers, ensuring the integrity of the training data without sacrificing clean samples. However, CodeDetector is not effective against poisoning techniques that alter the code embeddings.

3. Data poisoning

poisoned samples are malicious inputs that maintain their original structure while being altered just enough to manipulate the model's learning process. By including these poisoned samples during training, they skew the model's understanding, leading to systematic errors during inference.

3.1. Code poisoning techniques

There are various algorithms for generating poisoned samples in the NLP field, but many of them are less effective when applied to source code for several reasons. First, code has a distinct syntax and structure, differing from natural language text in terms of tokenization, grammar, and programming constructs. Second, the code relies on a specialized vocabulary containing programming keywords, function names, and variable names, making it challenging for poison attacks to generate meaningful perturbations.

In general, code poisoning techniques adopt two distinct transformation strategies, aiming to alter the structure or behavior of source code without compromising its fundamental functionality. These strategies are "identifier renaming" and "dead-code insertion". The former changes the names of variables or method parameters, while the latter inserts code that does not affect the source code snippet. In the following, we explain four poisoning techniques that implement these strategies.

3.1.1. DAMP

DAMP (Yefet et al., 2020) is a novel technique for generating poisoned samples to attack neural models of code. It aims to find semantically equivalent program variants that cause a model to misclassify. Using gradient-guided transformations, such as renaming variables and adding unused code, DAMP intelligently explores the space of valid program transformations. The process involves calculating gradients concerning discrete program elements, selecting the most impactful transformation, and applying it iteratively until the desired misclassification is achieved. DAMP supports both targeted and non-targeted attacks and operates under a white-box setting with access to model gradients.

3.1.2. MHM

MHM (Zhang et al., 2020) is an algorithm designed to generate poisoned samples for source code processing models through iterative identifier renaming while preserving the code's functionality. Based on the Metropolis–Hastings sampling method, an MCMC approach, MHM operates in stages: selecting a source identifier, proposing a new target identifier, and determining whether to accept or reject the change based on the model's classification probabilities. The generated examples maintain lexical, grammatical, and syntactical correctness and compile successfully. By repeating this process over several iterations, MHM efficiently generates poisoned samples while respecting the structural constraints of programming languages.

3.1.3. ALTER

ALTER (Yang et al., 2022) is a method for generating poisoned samples aimed at pre-trained models of code, with an emphasis on producing examples that appear natural to human developers. ALERT uses a masked language model, such as CodeBERT, to suggest contextually appropriate variable substitutions. It ranks candidate replacements based on their semantic similarity to the original tokens and selects the top-k most natural options. ALERT employs a two-step search: a Greedy-Attack that prioritizes substituting the most important variables to reduce the model's confidence, followed by a genetic algorithm-based GA-Attack if needed. The approach balances operational and natural semantics, producing more human-like poisoned samples while achieving high attack success rates against models like CodeBERT.

3.1.4. TextFooler

TextFooler (Jin et al., 2019) is a framework for generating poisoned samples to attack natural language processing models in a black-box setting, where the attacker has no access to the model's architecture or parameters. It operates in two steps: first, it ranks words based on their importance to the model's prediction by measuring how much the prediction changes when each word is removed. Then, in the Word Transformer step, it replaces important words with semantically similar synonyms that maintain the sentence's meaning and grammatical correctness. The process continues until the model's prediction is altered or all options are exhausted, aiming to minimize word changes. TextFooler has proven effective in tasks like text classification and textual entailment, significantly reducing model accuracy while modifying less than 20% of the original words. Its key innovation lies in generating adversarial text examples that preserve both meaning and grammaticality, overcoming the challenges posed by the discrete nature of text.

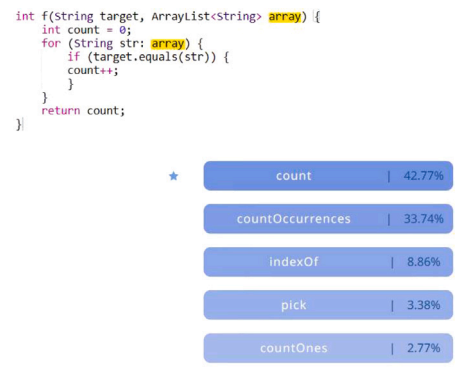
We investigated the potential of TextFooler in source code poisoning. We adapt its ability to generate semantically similar adversarial examples to introduce subtle and undetectable perturbations in source code. In particular, we employ TextFooler to change method names, method parameters, and variable names in source code, rather than focusing on Java-specific keywords. This keeps the overall structure of the code intact (ie., preserves functionality) but alters the embeddings. We refer to this adaptation of TextFooler, tailored specifically for source code poisoning, as CodeFooler. Unlike the other techniques, CodeFooler generates poisoned samples that appear abnormal and more suspicious to humans such as those with long variable names or meaningless identifiers.

3.2. Motivating example

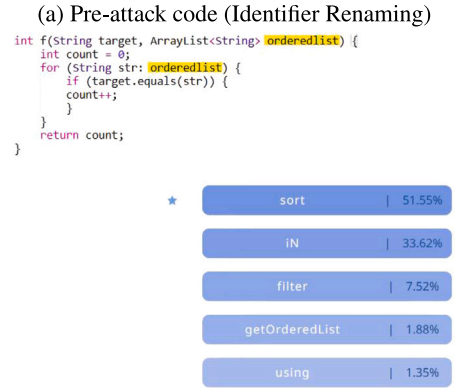
Figs. 1 and 2 illustrate two examples. In the first example, the attacker uses the identifier renaming poisoning strategy to change the identifier "array" to "ordered list", which causes the Code2Vec prediction to switch from "count" to "sort". This type of attack exploits the fact that the model relies on the semantics of the code identifiers to make predictions. Fig. 1(a) presents a clean source code sample along with its corresponding predictions generated by Code2Vec, and Fig. 1(b) presents a code sample after it has been poisoned.

In the second example, the attacker introduces dead code "int introsorter = 0" to the input data, which does not affect the functionality of the code, but changes the Code2Vec prediction from "indexOf" to "sort". This type of attack exploits the fact that the model may not fully understand the structure and semantics of the code, and can be tricked by seemingly innocuous changes. Fig. 2(a) presents a clean source code sample along with its corresponding predictions generated by Code2Vec and Fig. 2(b) presents the code sample after it has been poisoned.

> *Existing techniques depend merely on the source code itself and fall short when poisoned samples appear normal, but their underlying embeddings have been altered. Hence, we need a protection technique that can reveal poisoned samples from code embeddings.*

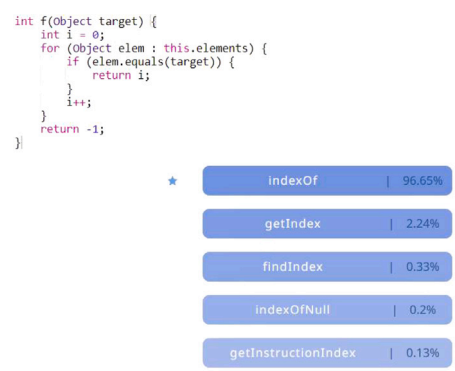


(a) Pre-attack code (Identifier Renaming)

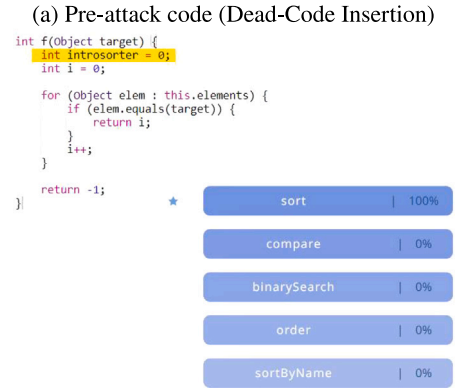


(b) Post-attack (Identifier Renaming).

Fig. 1. Comparison of pre- and post-attack code with Code2Vec predictions for identifier renaming.



(a) Pre-attack code (Dead-Code Insertion)



(b) Post-attack code (Dead-Code Insertion)

Fig. 2. Comparison of pre- and post-attack codes with Code2Vec predictions for dead-code insertion.

4. Data preparation

Fig. 3 illustrates our data preparation pipeline, which we discuss in the following.

4.1. Poison datasets

We employed the CoDesc dataset (BUET CSE NLP Group, 2025), a large-scale noise-filtered dataset of source code and its natural language descriptions, focusing on Java which has a widespread use in deep learning for coding tasks. We selected 274,000 distinct samples from CoDesc and created two datasets namely "PrimarySet" and "UnseenSet". The collection of distinct samples was necessary as some techniques may generate identical poisoned samples.

PrimarySet includes 250,000 samples. We divided this set into two equal parts, each containing 125,000 samples. The first half consists of clean source code samples, while the second half is further divided into four equal, distinct parts, each containing 31,250 samples. We applied one poisoning technique — DAMP, MHM, ALTER, and CodeFooler — to each of these four parts to generate poisoned samples. This is the primary dataset.

UnseenSet includes 24,000 samples. We poisoned 3000 samples using each of the four techniques and combined each part with 3000 clean samples to create a dataset of 6000 samples per technique. Poisoned samples for DAMP (Tech SRL, 2023), MHM, and ALTER (Soarsmu, 2023) were generated using the respective GitHub repositories. The TextAttack library (QData, 2023) was used to generate poisoned samples with CodeFooler.

4.2. Feature extraction

We applied Code2Vec, CodeBERT, and FastText to each sample to generate embeddings. These models have demonstrated reliability, efficiency, and effectiveness in recent source code analysis tasks such as code completion, summarization, and defect detection (Sahar et al., 2024; Dou et al., 2024).[2]

4.2.1. Code2Vec

Code2Vec (Alon et al., 2018) is a neural model that converts code snippets into fixed-length vectors known as "code embeddings". The process begins by transforming a code snippet into an Abstract Syntax Tree (AST), which captures the syntactic structure of the code. Each AST node represents a syntactic element, like a variable or function. Code2Vec decomposes the AST into paths from root to leaf nodes and assigns a vector to each path, encoding its syntactic and semantic properties. Through attention mechanisms, the model aggregates these paths into a single vector representing the entire snippet. This "code vector" is then used to predict semantic properties such as function names, return types, or arguments. The model generates a ranked list of top-10 predictions for the snippet, along with confidence scores for each suggestion, reflecting its certainty.

4.2.2. CodeBert

CodeBERT (Feng et al., 2020), a pre-trained language model, bridges the gap between natural and programming languages. Trained on a large dataset of code and natural language pairs, CodeBERT understands both the syntax and semantics of code. It is applied in tasks such as code searching, documentation generation, and code summarization. To generate code embeddings for Java code, CodeBERT tokenizes the code snippet into individual tokens. Each token is then assigned an embedding that captures its meaning and context. Positional encoding is added to represent token order, allowing CodeBERT to understand relationships between tokens. These token embeddings, combined with positional encodings, are processed by a Transformer encoder to capture long-range dependencies. The final result is a 768-dimensional vector representation of the code snippet.

There exist more recent language models, such as Codex and CodeT5+. Nevertheless, we adopted CodeBERT for two main reasons. First, it is a well-known and widely used model in this field, offering extensive tool and community support, along with pre-processing pipelines that simplify implementation and accelerate project development. Second, it is a computationally efficient option that strikes a good balance between performance and resource requirements, making it more practical than larger, resource-intensive models.

4.2.3. FastText

FastText (Bojanowski et al., 2017) model is a word-embedding technique used in natural language processing (NLP). FastText maps words into a fixed-size vector space using a hashing trick, where words are represented by averaging the embeddings of their constituent character n-grams, computed through hash-based indices. This average vector undergoes a linear transformation, producing the final word representation as a continuous vector in a high-dimensional space. Training the FastText model from scratch, instead of using a pre-trained version, optimizes word embeddings for the specific task, capturing domain-specific nuances and code-related terminology.

Textual data requires a series of preprocessing steps, which involves lowercasing, tokenization, removing special characters and punctuation, eliminating stop words, stemming, removing HTML tags and URLs, and omitting rare words. Important identifiers in the source code shall remain intact, preventing any modifications that could make the code uncompilable.

4.3. Final datasets

Deep learning models operate on numerical data, so any textual input, including code, must be transformed into numerical representations. To achieve this, we feed code snippets directly into Code2Vec and CodeBERT to obtain embeddings. We pass the textual data from Code2Vec i.e., function name to FastText for textual embedding. We also preprocess code snippets before feeding them into FastText for sentence-level embeddings.

Table 1 presents an overview of the sample distribution in the final datasets. The primary set contains 250,000 samples, 125,000 clean and 125,000 poisoned samples, where each poison technique contributed 31,250 samples. We divided this dataset into 80% for training, 10% for validation, and 10% for testing. The unseen set includes 24,000 samples, evenly split between 12,000 clean and 12,000 poisoned samples, where 3000 poisoned samples exist per each poison technique.

Table 1
The distribution of poison and clean samples in final datasets.

	Primary set	Unseen set
Clean Sample	125,000	12,000
DAMP	31,250	3,000
MHM	31,250	3,000
ALTER	31,250	3,000
CodeFooler	31,250	3,000
Total	250,000	24,000

5. The CodeGarrison model

We developed CodeGarrison (CG), a hybrid model designed to detect poisoned samples in code-related tasks by utilizing code embeddings. Acting as a preprocessing step, CG identifies and removes poisoned samples from the dataset, ensuring that only clean data is used

[2] We could not explore large language models (LLMs) like GPT-4 and CodeT5+ due to computational constraints, but this remains a promising direction for future research.

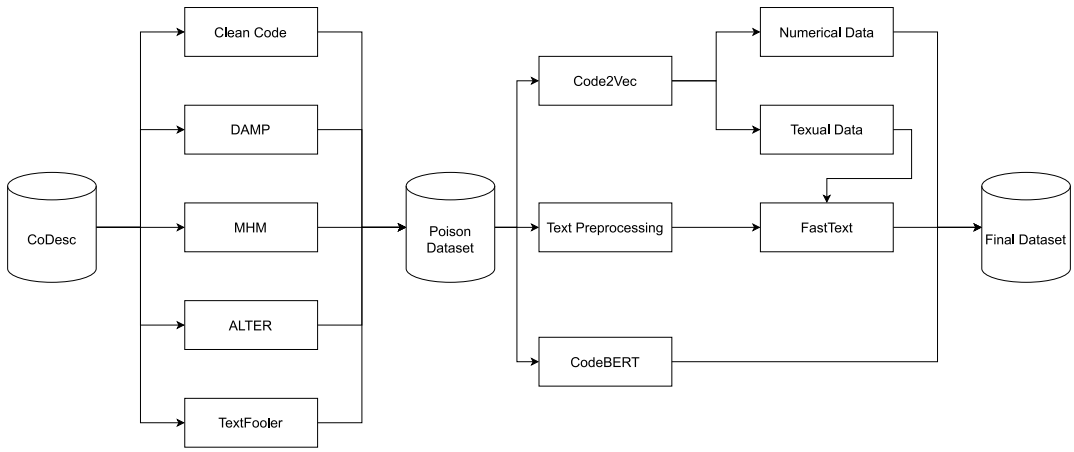


Fig. 3. Data preparation pipeline.

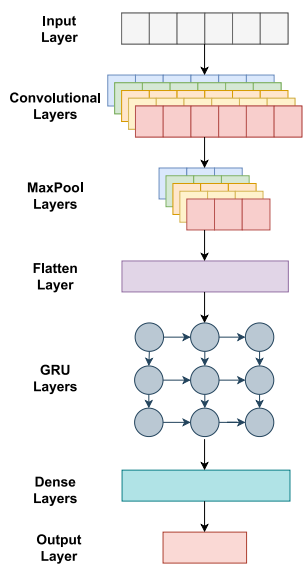


Fig. 4. The model architecture.

for training. CG's hybrid architecture combines multiple embeddings to enhance flexibility and overcome the limitations of any single model. Specifically, it leverages Code2Vec for capturing syntactic information through AST-based embeddings, CodeBERT for semantic understanding with transformer-based embeddings, and FastText for efficiently handling variations at the token level. This comprehensive approach allows CG to effectively detect and filter out poisoned samples before training begins.

We provide a comprehensive overview of CG's architecture, explain our training strategy, and present the results at the end.

5.1. Network architecture

The architecture of the CG model, shown in Fig. 4, consists of several components.

Input Layer: The input layer is the first layer of the network, in which raw data are introduced into the model for processing. The input data is represented as a one-dimensional array (i.e. a single vector), where the data points are sequentially ordered. Transforming sequential code into a single vector is a powerful technique that serves as a form of dimensionality reduction or feature extraction and offers several advantages. By condensing data into a single vector, the most important features of the sequence are emphasized, improving the computational efficiency and preventing overfitting.

Convolutional Neural Network (CNN) Layers: The CNN employs layers for data feature extraction, reducing parameters through local perception, and weight sharing to enhance the model efficiency. Multiple convolution kernels form a convolution and extract data features; however, this often increases the feature dimension (Peng et al., 2018). We use a pooling layer to mitigate this problem. In particular, we apply maximum pooling that samples the convolution result and reduces the vector size to prevent overfitting (Rawat and Wang, 2017).

Flatten Layer: The flattened layer is used to flatten the output of the CNN layer into a one-dimensional vector.

Gated Recurrent Unit (GRU) Layers: GRU, a variant of RNNs, addresses the challenges of vanishing and exploding gradients in traditional RNNs by incorporating gating mechanisms. The key components of the GRU are the update (z_t) and reset gates (r_t). Fig. 5 illustrates the GRU architecture. The computations involve the update gate (z_t) and reset gate (r_t), which are applied to each element in the input sequence and control the flow of information. The update gate controls how much past information is retained, while the reset gate determines how much past information is forgotten. The GRU layers sequentially generate hidden states, enabling the model to capture both short-term patterns and long-term dependencies.

In our architecture, we stacked multiple GRU layers to handle input sequences, allowing the model to learn hierarchical features that capture both detailed and abstract patterns in the data (Chung et al., 2014).

The input to the GRU layer is a flattened vector obtained from the output of the CNN layer, treated as a sequence of values representing features extracted by the CNN. The output of the GRU layer is a sequence of hidden states, each containing information about the input sequence up to that point. The final hidden state consolidates the entire input sequence, which is essential for capturing nuanced and abstract patterns. This representation is then fed into a dense layer to make predictions or classifications based on learned hierarchical features.

Although GRUs are designed to handle sequences, using fixed-size vectors as inputs can simplify the model's training process, leading to more stable and faster training owing to the consistent input size. This transformation process is akin to embedding techniques, such as Code2Vec, CodeBERT, and FastText, where high-dimensional or sequential data are transformed into dense vector representations that preserve the semantic and structural properties of the original data. Assuming that a single vector effectively encapsulates the critical features of the sequence, the GRU can still leverage its ability to detect temporal or sequential patterns through hidden state transitions. Thus, the initial

Table 2
Hyperparameters for the CG model.

Hyperparameter	Description	Value
Hidden State Sizes of GRU Layers	Internal memory cell dimensions	[256, 256, 256]
Dense Layer Sizes	Fully connected layer dimensions	[1024, 512]
Convolutional Channel Sizes	Feature extraction layer dimensions	[16, 32, 64, 128, 256]
Kernel Size	Size of convolutional filter	3
Padding Size	Padding added to input	1
Learning Rate	Step size for optimization	0.001
Scheduler	Learning rate adjustment strategy	Cosine Annealing
Max Gradient Norm	Gradient clipping threshold	1
L1	L1 regularization strength	1×10^{-4}
L2	L2 regularization strength	1×10^{-4}
Dropout Rate	Neuron deactivation probability	0.3

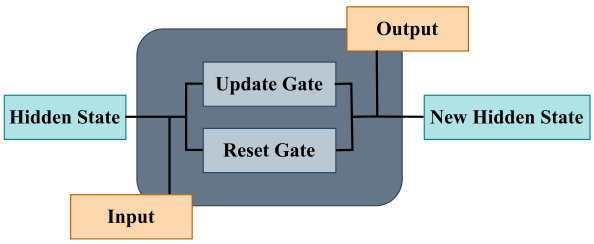


Fig. 5. GRU's standard architecture.

vector serves as a starting point for the GRU to refine and build upon, balancing simplicity with the preservation of meaningful features.[3]

Dense Layers: These dense layers serve to reduce dimensionality, which is critical when dealing with high-dimensional GRU outputs, thereby making the network more computationally efficient. They introduce nonlinearity through the activation function, enhancing the capacity of the model to capture complex patterns in the input data. In addition, dense layers allow for feature transformation, creating more task-specific representations, which can be pivotal in emphasizing relevant features and suppressing noise. Finally, they prepare the output for the final layer, which is, in our case, a classification layer, where they map intermediate representations to the appropriate number of output classes (clean or poisoned samples).

Output Layer: The final layer of our model is the output layer with a softmax activation function. The softmax function converts the network output into a probability distribution over possible classes.

5.2. Training

We explain our hyperparameter tuning, the loss function, the optimization algorithm, and the training procedure.

Hyperparameter Tuning: We optimized our model by combining Optuna's automated features with hands-on trial-and-error adjustments.[4] Initially, Optuna explored a predefined range of important hyperparameters, such as layer count, hidden sizes, channels, kernel sizes, padding, learning rate, scheduler, gradient norm, L1 and L2 regularization, and dropout rate. Automated optimization provided an initial set of promising hyperparameters. Moving on to manual fine-tuning, we systematically tweaked the individual hyperparameters based on the observed performance on a validation set.

Table 2 lists the final set of hyperparameters used in our model.

Loss Function: The cross-entropy loss is a key loss function for classification tasks with two or more classes. It is used to optimize the predicted class probabilities to match the true labels. The loss functions well with softmax activation, handles uncertainty, and has smooth gradients for optimization. Its established success and versatility make it a common and reliable choice for classification.

Optimization Algorithm: We use the Adam optimization algorithm, complemented by Weight Decay to prevent overfitting through parameter regularization. Gradient Clipping to limit the magnitude of gradients and Learning Rate Scheduling is integrated to enhance convergence and enhance the final results.

Training Procedure: The training process spans 256 epochs and stops when the validation loss ceases to exhibit significant improvement, thereby leveraging the early stopping mechanism. For the hidden state sizes of the GRU Layers, we use dimensions of [256, 256, 256], aiming to capture intricate temporal dependencies within the data. The dense layer sizes are set to [1024, 512], defining the dimensions of the fully connected layers for complex feature extraction and representation. In addition, convolutional channel sizes of [16, 32, 64, 128, 256] are employed to facilitate hierarchical feature extraction through convolutional layers. The kernel size for the convolutional filters is set to 3, and a Padding Size of 1 is added to the input to maintain spatial information. To further enhance the model's generalization capabilities, dropout is strategically employed. At a rate of 0.3, neurons are randomly deactivated during training, promoting robustness and reducing dependency on specific nodes. Furthermore, both L1 and L2 regularization strengths are set to 1×10^{-4}, discouraging the model from relying on individual features and mitigating the risk of overfitting. Additionally, gradient clipping is applied at a threshold of 1 to limit the magnitude of the gradients. This measure ensures training stability and prevents potential gradient-related issues during the optimization process.

5.3. Results

Table 3 presents performance metrics for the CG model across training, validation, and test sets, with labels (Clean, Poison) as the target variable. The accuracy of the model is consistently high, reaching 95.0% on the training set and maintaining a strong performance on the validation and test sets at 94.0% and 94.0%, respectively. Precision, which measures the ratio of correctly predicted positive observations to the total predicted positives, is exceptionally high across all sets, at 97.0%, 96.0%, and 94.1%, respectively. However, the recall values, representing the proportion of actual positives correctly predicted, are slightly lower, with the training set achieving 94.2% and the validation and test set achieving 93.9% and 93.3%, respectively. The F1 score, a harmonic mean of precision and recall, also exhibits consistently strong results across the three sets with values of 95.5%, 95.1%, and 93.7%, respectively. Overall, the model demonstrates robust performance with high accuracy and precision, although there is a slight trade-off in recall, indicating potential room for improvement in capturing true-positive instances. The inclusion of metrics from the training, validation, and test sets provides a more comprehensive evaluation of the model. Training metrics help detect overfitting by comparing

[3] We tested RNN and LSTM architectures as well, but the GRU architecture outperformed both in accuracy and training efficiency. GRU's ability to address the vanishing gradient problem, coupled with its simpler structure, likely accounts for this superior performance.

[4] https://optuna.org/

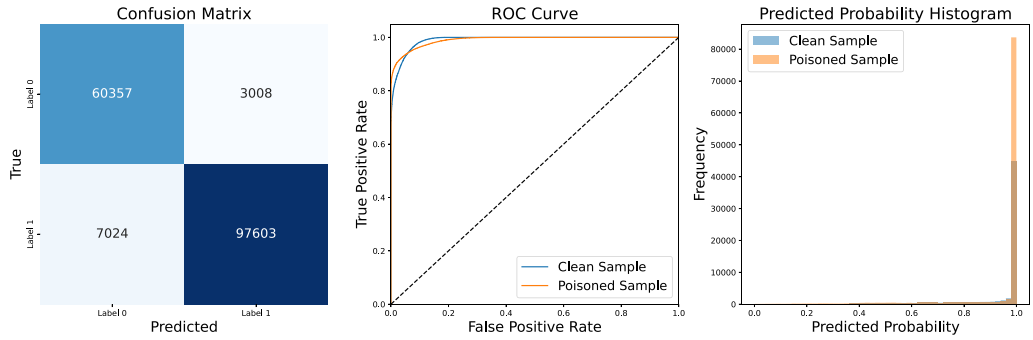


Fig. 6. Confusion matrices, ROC curve, and probability histograms.

Table 3
The performance metrics for the training, validation, and test data.

Metric	Train	Validation	Test
Accuracy	0.950	0.940	0.940
Precision	0.970	0.960	0.941
Recall	0.942	0.939	0.933
F1 Score	0.955	0.951	0.937

performance against validation and test sets, while validation set metrics guide model tuning. Test set metrics ultimately assess the model's ability to generalize to unseen data, offering a complete view of its effectiveness.

The confusion matrix in Fig. 6 (left) shows the performance of the two-class classification model. It shows 19,249 true negatives and 18,335 true positives, highlighting the model's strong accuracy in predicting both negative and positive instances. However, there are 751 false positives and 1665 false negatives, suggesting the need for improvement to minimize errors and enhance the overall performance.

Fig. 6 (middle) shows the ROC curve. The curve of the clean sample indicates better discrimination ability with higher sensitivity and specificity than the poisoned sample. The poisoned sample's curve shifts to the left, suggesting a higher false positive rate (lower specificity) at similar true positive rates (sensitivity), indicating a higher likelihood of incorrect positive classifications.

The predicted probability histogram, shown in Fig. 6 (right), depicts the confidence of the model in binary classification. For samples labeled 1, most receive high predicted probabilities near 1.0, signifying strong confidence. However, a few instances show lower confidence around 0.5 or 0.0. In contrast, label 0 samples display a broader range of predicted probabilities (0.5 to 1.0), indicating variability and lower overall confidence in the model's predictions for this class.

6. Evaluation

We compare the performance of CG against the state-of-the-art model. We assess CG's resilience against new attacks. In the end, we investigate features that are important to uncover poisoned samples. The results presented in this section represent the optimal values.

6.1. Performance comparison

ONION is a state-of-the-art poisoned sample detection technique (Qi et al., 2021). It detects unnatural words (triggers) by using a pre-trained language model and a leave-one-out strategy, which removes suspect words and checks if this reduces the perplexity of the sequence. If removing a word significantly decreases perplexity, it is flagged as a potential trigger.

To use ONION, the process involves two main steps: training a poisoned victim model and testing ONION's defense effectiveness. In the first step, we utilize CodeBERT as the victim model, fine-tuning it on our primary dataset to evaluate its susceptibility to backdoor attacks. In the second step, we employ CodeGPT as the language model to calculate perplexity scores, similar to the approach of a recent study (Li et al., 2024). ONION detects trigger words irrelevant to the context and removes them, which significantly reduces the perplexity of the entire code snippet.[5]

We evaluate the performance of the CG model against ONION across four attack scenarios: DAMP, MHM, ALERT, and CodeFooler. Table 4 presents the results. The CG model outperforms ONION in all four attack scenarios. The MHM attack excels in accuracy, precision, recall, and F1 score, showcasing its proficiency in identifying and classifying poisoned samples. The DAMP attack also shows that our CG model outperforms ONION, particularly excelling in the recall, indicating its capability to correctly identify a high proportion of instances of this attack type. Similarly, in the ALERT attack, our model maintains an advantage across all metrics, with higher precision, suggesting a better ability to minimize false positives. Although our model still leads to accuracy and precision in the CodeFooler attack. On the other hand, ONION effectively detects unnatural words in input sequences by utilizing perplexity to identify these unnatural words and considers them as inserted triggers. However, ONION preforms slightly worse than CG.

RQ1: Is CG effective in detecting poisoned code samples? We showed that CG effectively detects poisoned samples generated by DAMP, MHM, ALERT, and CodeFooler, with accuracy rates of 93.1%, 95.5%, 94.9%, and 90.3%, respectively. It significantly outperforms the ONION model, which achieved much lower accuracy rates of 48.7%, 48.3%, 41.9%, and 54.0% for the same attacks.

6.2. New attack protection

We retrained CG on the primary dataset across four separate iterations, each time excluding samples tied to a specific attack technique, namely DAMP, MHM, ALTER, and CodeFooler. We relied on the unseen set of 24000 samples to evaluate CG's ability to detect unseen attacks. For each technique, the dataset included 3000 poisoned and 3000 clean samples, yielding 6000 test samples per technique. We tested each model exclusively on the poisoned samples from the excluded technique, allowing us to assess the model's capability to detect new poison attacks. CG achieved an average accuracy of 85.6% across the four poisoning techniques.

Table 5 presents the evaluation results for each one. The CG model, when not exposed to the DAMP samples during training, demonstrates a

[5] We were eager to compare CG with CodeDetector (Li et al., 2024), a more recent technique than Onion. Even though CodeDetector is ineffective against poisoning attacks that modify code embeddings, it still represents an advancement. Unfortunately, the model is not publicly available, and despite reaching out to the authors several times for access to the model or source code, we received no response.

Table 4
Evaluation results for different attack scenarios.

Metric	CG Model				ONION			
	DAMP	MHM	ALERT	CodeFooler	DAMP	MHM	ALERT	CodeFooler
Accuracy	0.931	0.955	0.949	0.903	0.911	0.908	0.911	0.893
Precision	0.934	0.937	0.937	0.895	0.911	0.908	0.912	0.898
Recall	0.928	0.974	0.968	0.912	0.911	0.908	0.911	0.911
F1 Score	0.931	0.955	0.950	0.903	0.911	0.898	0.911	0.891

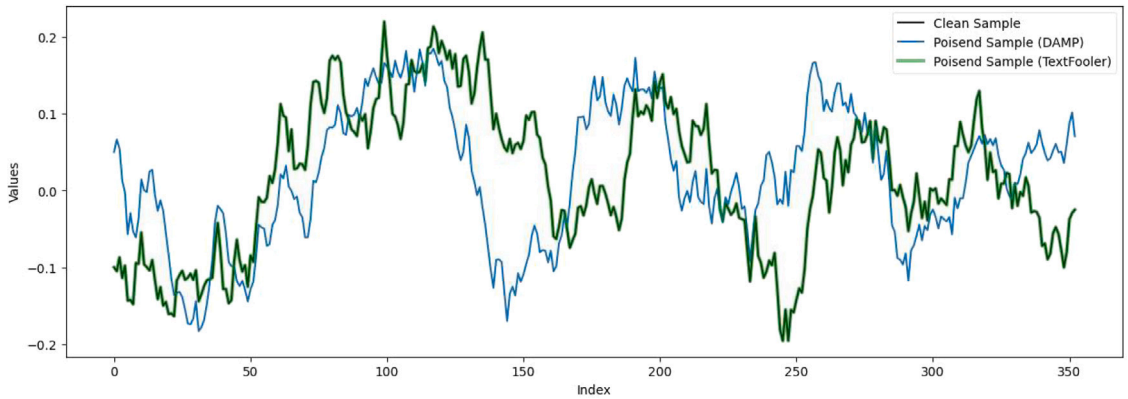


Fig. 7. The comparison of Code2Vec vectors between clean samples and poisoned samples from DAMP and CodeFooler.

Table 5
Model evaluation on unseen attacks.

Metric	No DAMP	No MHM	No ALERT	No CodeFooler
Accuracy	0.925	0.951	0.938	0.609
Precision	0.938	0.935	0.933	0.582
Recall	0.911	0.968	0.943	0.770
F1 Score	0.924	0.951	0.938	0.663

Table 6
Performance metrics for different features.

Model	Accuracy	Precision	Recall	F1
All Features	0.940	0.941	0.933	0.937
Only Embeddings	0.950	0.958	0.957	0.958
CodeBERT Embeddings	0.925	0.900	0.957	0.928
Code2Vec Embeddings	0.816	0.753	0.942	0.837
FastText Embeddings	0.792	0.729	0.929	0.817

high accuracy of 92.5%. Similarly, when not trained on MHM samples, the CG model shows high adaptability with an accuracy of 95.1%, indicating its robustness against MHM attacks. In the absence of ALERT samples, the CG model maintains a strong performance of 93.8%. However, without CodeFooler samples, the model faces a more significant challenge, resulting in a lower accuracy of 68.6% and limitations in recall and F1 Score, suggesting difficulties in handling poison examples generated by CodeFooler.

The CG model faces significant challenges in detecting poisoned samples generated by CodeFooler, as indicated by the lower accuracy of 68.6%. This issue arises because poisoning techniques like DAMP, MHM, and ALERT introduce changes to the code's embedding representations, making detection easier. In contrast, CodeFooler's alterations do not significantly impact the embeddings, resulting in difficulty for the model, which relies on these embeddings for classification. To illustrate the matter, Fig. 7 provides an embedding comparison between clean code and poisoned samples of DAMP and CodeFooler. The DAMP-poisoned sample shows clear differences in embeddings compared to the clean sample, while the CodeFooler-poisoned sample's embeddings remain nearly identical to the clean code. This explains the CG model's difficulty in identifying CodeFooler-generated poison, as the embeddings fail to capture meaningful differences.

RQ2: How does CG perform against unseen poison attacks? We found that CG offers a promising generalization performance with an average accuracy of 85.6%. In particular, CG detected samples poisoned with unseen attacks with accuracy rates of 92.5% (DAMP), 95.1% (MHM), 93.8% (ALERT), and 60.9% (CodeFooler).

6.3. Important features

We created the Saliency maps, shown in Fig. 8, to visualize key features impacting CG's prediction. The most important features are where the amplitude of the spikes begins to increase (i.e., near the end of the x-axis). These features include Code2Vec embeddings, FastText embeddings, and CodeBERT embeddings. The top ten Code2Vec predictions and their scores had little impact on the model's decision-making.

We investigated the impact of each feature set to understand how much they contribute to CG's performance. In addition to the investigation of all features that we presented earlier, we trained and tested the model with the following distinct feature sets namely Code2Vec embeddings, FastText embeddings, CodeBERT embeddings, and the combination of the three embeddings. Table 6 presents the result.

We found that CG performs best when using all embeddings together (i.e., only embeddings), achieving an accuracy of 95.0%, 95.8% precision, 95.7% recall, and an F1 score of 95.7%. However, when additional features from Code2Vec were included (i.e., all features), the performance metrics slightly dropped, with accuracy decreasing to 94.0%, precision to 94.1%, recall to 93.3%, and the F1 score to 93.7%. The CodeBERT embedding model performs well across all metrics, with an accuracy of 92.5%, precision of 90.0%, recall of 95.7%, and an F1 score of 92.8%. In contrast, the Code2Vec embedding model, while showing a lower accuracy of 81.6% and precision of 75.3%, demonstrates a higher recall of 94.2% and an F1 score of 83.7%. Similarly, the FastText embeddings model achieves a strong recall of 92.9%, but with lower accuracy (79.2%) and precision (72.9%), resulting in an F1 score of 81.7%. These results underscore how the choice and combination of embeddings can significantly influence the model's overall performance.

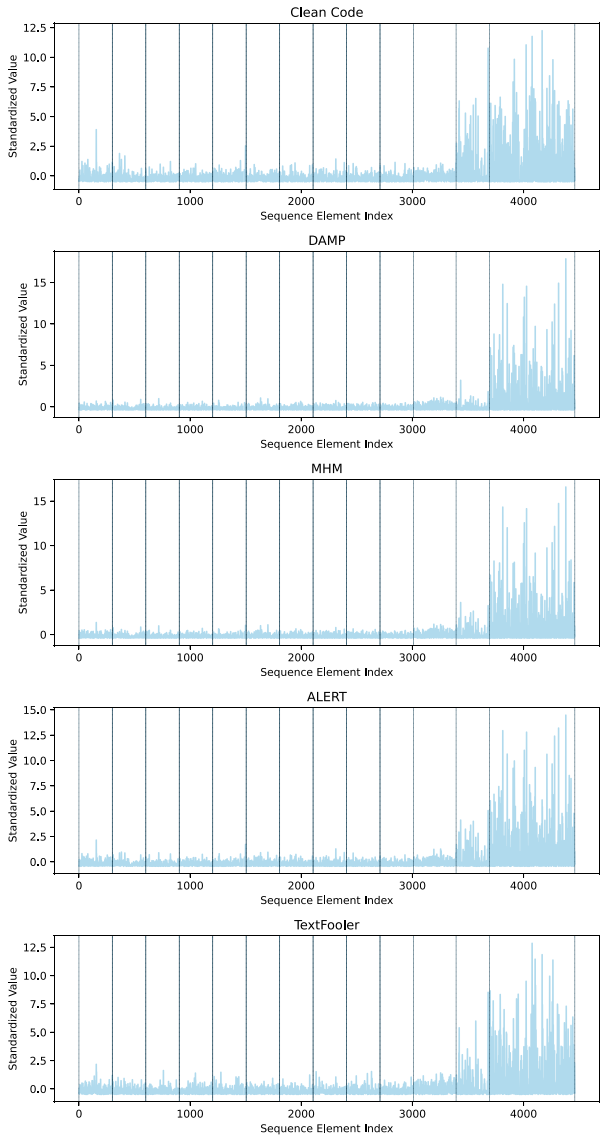


Fig. 8. Saliency map highlighting key features. The x-axis represents the dimensions of the feature vector, and the y-axis denotes the magnitude of the importance or saliency score for each dimension.

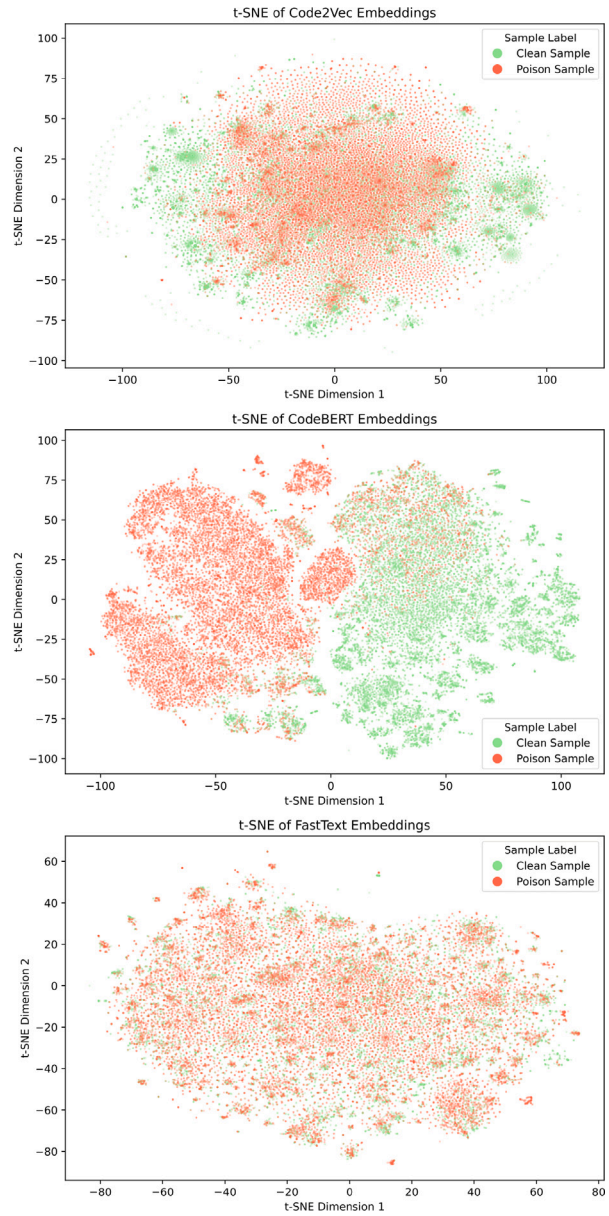


Fig. 9. t-SNE Output.

We used a t-SNE technique to further explore the impact of important features. The outcome, illustrated in Fig. 9, revealed that CodeBERT embeddings achieve the clearest separation between clean and poisoned samples, followed by Code2Vec and FastText. This separation indicates that the clean and poisoned samples occupy different regions in feature space, allowing the classification model to differentiate between them effectively. These observations align with CodeBERT's superior classification accuracy, followed by Code2Vec and FastText.

RQ3: How does each feature impact the CG's ability to differentiate between poisoned and clean samples? The study demonstrates that using a combination of embeddings from Code2Vec, CodeBERT, and FastText (i.e., only embeddings) yields the best performance, achieving an overall accuracy of 95.0%, which is higher than all features (i.e., when Code2Vec name predictions and scoring were included as well). When we used each embedding alone, the accuracy dropped to 92.5%, 81.6%, and 79.2% for CodeBERT, Code2Vec, and FastText, respectively.

7. Threats to validity

There are several threats to validity of this study that we explain in the following.

Construct Validity: Our study focuses on the Java programming language due to the poisoned sample generation techniques employed. This specificity introduces a potential threat to construct validity, as our methodology may capture Java-specific characteristics rather than accurately representing the broader concept of poisoned code detection across different programming languages.

Internal Validity: The effectiveness of our experimental results depends on several critical hyperparameter configurations. Factors such as input length, standardized at 1024 tokens for CodeGPT, and the number of training epochs play a vital role in the experiment's outcome. Additionally, the JavaBERT pre-trained model's architecture is designed to manage up to 512 subwords, which distinctly influences the unique characteristics and robustness of our experiment. These configurations underscore the importance of maintaining internal consistency to ensure the integrity of the experiment.

External Validity: Our findings might not apply universally to alternative attack methods, as we based our study on four existing models to generate poisoned samples. The specificity to these models raises concerns about the generalizability of our approach. Although our standardization efforts enhance performance within our specific context, this specificity might limit the applicability of our results to other scenarios or languages. As such, external validation would require extensive testing across diverse platforms and configurations to confirm the broader applicability of our findings.

8. Conclusion

We present CG, a hybrid deep-learning model designed to detect poisoned source code samples. We compared CG with the state-of-the-art approach ONION and found that CG significantly outperformed ONION. We also found that CG has a promising performance in uncovering new poison attacks. CG can be integrated into development pipelines either as a preprocessing module to clean poisoned samples from training datasets or as a real-time module within an IDE to flag potential poisoning triggers.

Incorporating attention mechanisms to identify important features in code embeddings and understand how specific parts of the input code influence model decisions offers a promising direction for future research. Moreover, as large language models (LLMs) like GPT-4 gain prominence, exploring their potential to generate and detect poisoned samples using zero-shot and few-shot learning presents another valuable avenue for investigation.

CRediT authorship contribution statement

Ehab Ghannoum: Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation. **Mohammad Ghafari:** Writing – review & editing, Validation, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data availability link is as follows: https://doi.org/10.5281/zenodo.14871552.

References

Aghakhani, H., Dai, W., Manoel, A., Fernandes, X., Kharkar, A., Kruegel, C., Vigna, G., Evans, D., Zorn, B.G., Sim, R., 2023. TrojanPuzzle: Covertly poisoning code-suggestion models. In: 2024 IEEE Sym-posium on Security and Privacy (SP). pp. 1122–1140, URL: https://api.semanticscholar.org/CorpusID:255522506.

Alon, U., Zilberstein, M., Levy, O., Yahav, E., 2018. Code2vec: Learning distributed representations of code. arXiv:1803.09473.

Bojanowski, P., Grave, E., Joulin, A., Mikolov, T., 2017. Enriching word vectors with subword information. arXiv:1607.04606.

Bruni, M., Gabrielli, F., Ghafari, M., Kropp, M., 2025. Benchmarking prompt engineering techniques for secure code generation with GPT models. In: Proceedings of the 2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering.

BUET CSE NLP Group, 2025. CoDesc dataset. URL: https://github.com/csebuetnlp/CoDesc. (Accessed 10 January 2025).

Carlini, N., Wagner, D., 2017. Adversarial examples are not easily detected: Bypassing ten detection methods. In: Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security. AISec '17, Association for Computing Machinery, pp. 3–14. http://dx.doi.org/10.1145/3128572.3140444, URL: https://doi.org/10.1145/3128572.3140444.

Chen, X., Salem, A., Chen, D., Backes, M., Ma, S., Shen, Q., Wu, Z., Zhang, Y., 2020. BadNL: Backdoor attacks against NLP models with semantic-preserving improvements. In: Proceedings of the 37th Annual ComputerSecurity Applications Conference. URL: https://api.semanticscholar.org/CorpusID:238354397.

Chung, J., Gulcehre, C., Cho, K., Bengio, Y., 2014. Empirical evaluation of gated recurrent neural networks on sequence modeling. ArXiv preprint arXiv:1412.3555.

Cotroneo, D., Improta, C., Liguori, P., Natella, R., 2024. Vulnerabilities in ai code generators: Exploring targeted data poisoning attacks. In: Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension. pp. 280–292.

Dou, S., Wu, Y., Jia, H., Zhou, Y., Liu, Y., Liu, Y., 2024. CC2Vec: Combining typed tokens with contrastive learning for effective code clone detection. Proc. ACM Softw. Eng. 1 (FSE), 1564–1584.

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., Zhou, M., 2020. CodeBERT: A pre-trained model for programming and natural languages. arXiv:2002.08155.

Firouzi, E., Ghafari, M., Ebrahimi, M., 2024. ChatGPT's potential in cryptography misuse detection: A comparative analysis with static analysis tools. In: Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement. ESEM '24, pp. 582–588. http://dx.doi.org/10.1145/3674805.3695408, URL: https://doi.org/10.1145/3674805.3695408.

Gu, T., Dolan-Gavitt, B., Garg, S., 2019. BadNets: Identifying vulnerabilities in the machine learning model supply chain. URL: https://arxiv.org/abs/1708.06733.

Ji, Y., Zhang, X., Ji, S., Luo, X., Wang, T., 2018. Model-reuse attacks on deep learning systems. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. URL: https://api.semanticscholar.org/CorpusID:53059573.

Jiang, N., Lutellier, T., Tan, L., 2021. Cure: Code-aware neural machine translation for automatic program repair. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering. ICSE, IEEE, pp. 1161–1173.

Jin, D., Jin, Z., Zhou, J.T., Szolovits, P., 2019. Is BERT really robust? Natural language attack on text classification and entailment. ArXiv preprint arXiv:1907.11932.

Jin, K., Zhang, T., Shen, C., Chen, Y., Fan, M., Lin, C., Liu, T., 2022. Can we mitigate backdoor attack using adversarial detection methods? IEEE Trans. Dependable Secur. Comput. 20 (4), 2867–2881.

Li, J., Li, Z., Zhang, H., Li, G., Jin, Z., Hu, X., Xia, X., 2024. Poison attack and poison detection on deep source code processing models. ACM Trans. Softw. Eng. Methodol. 33 (3), http://dx.doi.org/10.1145/3630008, URL: https://doi.org/10.1145/3630008.

Liu, Z., Qian, P., Wang, X., Zhuang, Y., Qiu, L., Wang, X., 2021. Combining graph neural networks with expert knowledge for smart contract vulnerability detection. IEEE Trans. Knowl. Data Eng. 35 (2), 1296–1310.

Lu, S., Guo, D., Ren, S., Huang, J., Svyatkovskiy, A., Blanco, A., Clement, C., Drain, D., Jiang, D., Tang, D., Li, G., Zhou, L., Shou, L., Zhou, L., Tufano, M., Gong, M., Zhou, M., Duan, N., Sundaresan, N., Deng, S.K., Fu, S., Liu, S., 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. arXiv:2102.04664.

Peng, H., Li, J., He, Y., Liu, Y., Bao, M., Wang, L., Song, Y., Yang, Q., 2018. Large-scale hierarchical text classification with recursively regularized deep graph-cnn. In: Proceedings of the 2018 World Wide Web Conference. pp. 1063–1072.

QData, 2023. TextAttack. URL: https://github.com/QData/TextAttack. (Accessed 3 February 2025).

Qi, F., Chen, Y., Li, M., Yao, Y., Liu, Z., Sun, M., 2021. ONION: A simple and effective defense against textual backdoor attacks. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, pp. 9558–9566. http://dx.doi.org/10.18653/v1/2021.emnlp-main.752, URL: https://aclanthology.org/2021.emnlp-main.752.

Rawat, W., Wang, Z., 2017. Deep convolutional neural networks for image classification: A comprehensive review. Neural Comput. 29 (9), 2352–2449.

Razmi, F., Xiong, L., 2023. Classification auto-encoder based detector against diverse data poisoning attacks. In: Data and Applications Security and Privacy XXXVII: 37th Annual IFIP WG 11.3 Conference, DBSec 2023, Sophia-Antipolis, France, July 19–21, 2023, Proceedings. Springer-Verlag, Berlin, Heidelberg, pp. 263–281. http://dx.doi.org/10.1007/978-3-031-37586-6_16, URL: https://doi.org/10.1007/978-3-031-37586-6_16.

Sahar, S., Younas, M., Khan, M.M., Sarwar, M.U., 2024. DP-CCL: A supervised contrastive learning approach using CodeBERT model in software defect prediction. IEEE Access.

Schuster, R., Song, C., Tromer, E., Shmatikov, V., 2020. You autocomplete me: Poisoning vulnerabilities in neural code completion. In: USENIX Security Symposium. URL: https://api.semanticscholar.org/CorpusID:220363858.

Shafahi, A., Huang, W.R., Najibi, M., Suciu, O., Studer, C., Dumitras, T., Goldstein, T., 2018. Poison frogs! targeted clean-label poisoning attacks on neural networks. In: Proceedings of the 32nd International Conference on Neural Information Processing Systems. NIPS '18, Curran Associates Inc., pp. 6106–6116.

Soarsmu, 2023. Attack pretrain models of code. URL: https://github.com/soarsmu/attack-pretrain-models-of-code. (Accessed 3 February 2025).

Steinhardt, J., Koh, P.W., Liang, P., 2017. Certified defenses for data poisoning attacks. In: Proceedings of the 31st International Conference on Neural Information Processing Systems. NIPS '17, Curran Associates Inc., pp. 3520–3532.

Sun, Z., Du, X., Song, F., Ni, M., Li, L., 2022. Coprotector: Protect open-source code against unauthorized training usage with data poisoning. In: Proceedings of the ACM Web Conference 2022. pp. 652–660.

Tech SRL, 2023. Adversarial examples. URL: https://github.com/tech-srl/adversarial-examples. (Accessed 3 February 2025).

Wan, Y., Zhang, S., Zhang, H., Sui, Y., Xu, G., Yao, D., Jin, H., Sun, L., 2022. You see what I want you to see: poisoning vulnerabilities in neural code search. In: Proceedings of the 30th ACM Joint EuropeanSoftware Engineering Conference and Symposium on the Foundations of Software Engineerign. URL: https://api.semanticscholar.org/CorpusID:253421850.

Wang, W., Li, G., Ma, B., Xia, X., Jin, Z., 2020. Detecting code clones with graph neural network and flow-augmented abstract syntax tree. In: 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering. SANER, IEEE, pp. 261–271.

Yang, Z., Shi, J., He, J., Lo, D., 2022. Natural attack for pre-trained models of code. In: Proceedings of the 44th International Conference on Software Engineering. ACM, URL: https://doi.org/10.1145%2F3510003.3510146.

Yang, Z., Xu, B., Zhang, J.M., Kang, H.J., Shi, J., He, J., Lo, D., 2024. Stealthy backdoor attack for code models. IEEE Trans. Softw. Eng. 50 (4), 721–741. http://dx.doi.org/10.1109/TSE.2024.3361661, URL: https://doi.org/10.1109/TSE.2024.3361661.

Yefet, N., Alon, U., Yahav, E., 2020. Adversarial examples for models of code. Proc. ACM Program. Lang. 4 (OOPSLA), http://dx.doi.org/10.1145/3428230, URL: https://doi.org/10.1145/3428230.

Zhang, H., Li, Z., Li, G., Ma, L., Liu, Y., Jin, Z., 2020. Generating adversarial examples for holding robustness of source code processing models. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, no. 01. pp. 1169–1176.

Ehab Ghannoum is a research assistant at TU Clausthal, Germany. He is highly interested in artificial intelligence and data science, and this work was conducted as part of his research track project in the Secure Software Engineering (SSE) Group. Ehab earned his M.Sc. in Computer Science from TU Clausthal in 2025 and his B.Sc. in Computer Engineering from Qatar University in 2020.

Mohammad Ghafari is Professor of Computer Science at TU Clausthal, where he leads the Secure Software Engineering (SSE) group. His research focuses on developing tools and techniques that facilitate secure software development. Mohammad earned his Ph.D. in Software Engineering from Politecnico di Milano in 2015.